RAPPORT D'ARCHITECTURE

UNIVERSITE GASTON BERGER

L'excellence au service du développement

CONCEPTION ET IMPLÉMENTATION D'UNE BASE DE DONNÉES DISTRIBUÉE RÉSILIENTE

Trinôme :

Fallou Sy , Khadidiatou NDIAYE, Ahmadou Tidiane Ndongo

Sommaire

1. Contexte et objectifs
2. Architecture
3. Stratégie de partitionnement
4. Modèle de données
5. Réplication et tolérance aux pannes
6. Analyse CAP / PACELC
7. Cohérence et anti-survente
8. Implémentation
9. Tests et résultats
10. Limites et perspectives
11. Conclusion

Résumé

GlobalShop, entreprise de commerce en ligne, subit des pannes en période de forte affluence et une latence élevée pour ses clients répartis sur quatre continents. Ce rapport présente un prototype de base de données distribuée qui répond à trois objectifs : **passer à l'échelle horizontalement**, **tolérer la perte de nœuds** et **garantir la cohérence du stock** (aucune survente).

L'architecture retenue repose sur **4 shards** (Afrique, Europe, Amérique, Asie), chacun composé d'un **primaire** et d'une **réplique** MySQL 8.4

(8 nœuds de données), et d'un **routeur** FastAPI qui oriente les requêtes, exécute les transactions, supervise le cluster et promeut automatiquement une réplique lorsqu'un primaire tombe. Le système est **CP**

(PACELC : **PC/EC**) : en cas de panne, un shard refuse les écritures plutôt que de risquer une incohérence. Le déploiement tient en une commande (docker compose up -d --build).

1. Contexte et objectifs

L'entreprise souhaite revoir l'architecture de son système de gestion des stocks et de passage de commandes. Deux problèmes sont identifiés : des **pannes récurrentes** lors des pics (Black Friday, soldes) et une **latence** pénalisante pour les clients éloignés du centre de données. Le cahier des charges impose au moins 4 nœuds interconnectés, une stratégie de partitionnement justifiée, un composant de routage, un positionnement explicite vis-à-vis du théorème CAP et la prévention de la survente en écriture concurrente.

Exigence	Réponse apportée
Mise à l'échelle horizontale	4 shards ; ajout de capacité par fragmentation des données (section 3)
Tolérance aux pannes	1 réplique par shard, failover automatique du routeur (section 5)
Cohérence du stock	un seul écrivain par produit + verrou de ligne + contraintes SQL (section 7)
Au moins 4 nœuds	8 nœuds MySQL (4 primaires + 4 répliques) + 1 routeur, en conteneurs Docker
Routage des requêtes	middleware Python/FastAPI, routage déterministe par ID ou par clé (section 2)

2. Architecture

Nous avons retenu l'**option A** du sujet : un middleware (Python/FastAPI) devant un SGBD relationnel (MySQL 8.4) par nœud. Ce choix donne un contrôle complet du routage, des transactions et du failover, et rend le comportement du système entièrement observable dans le rapport.

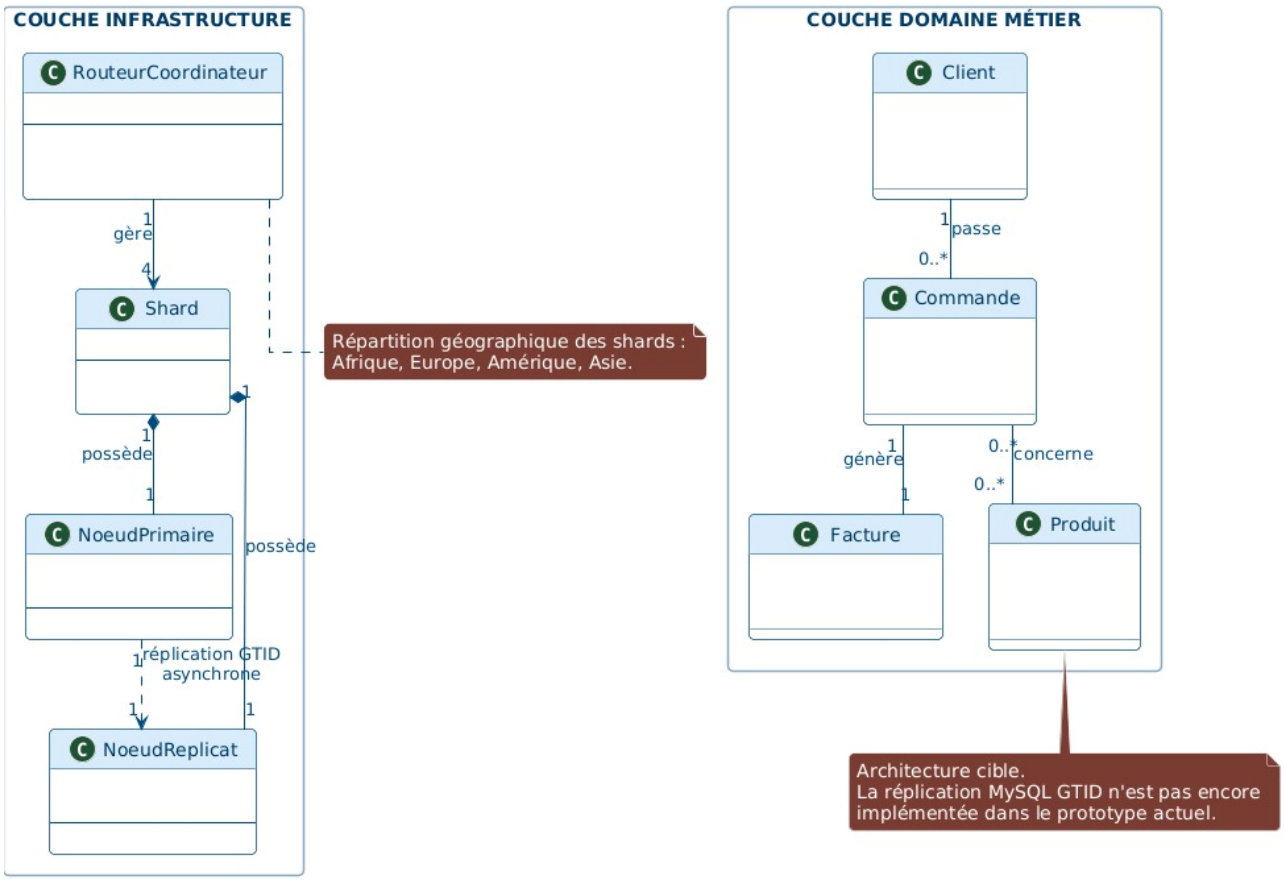
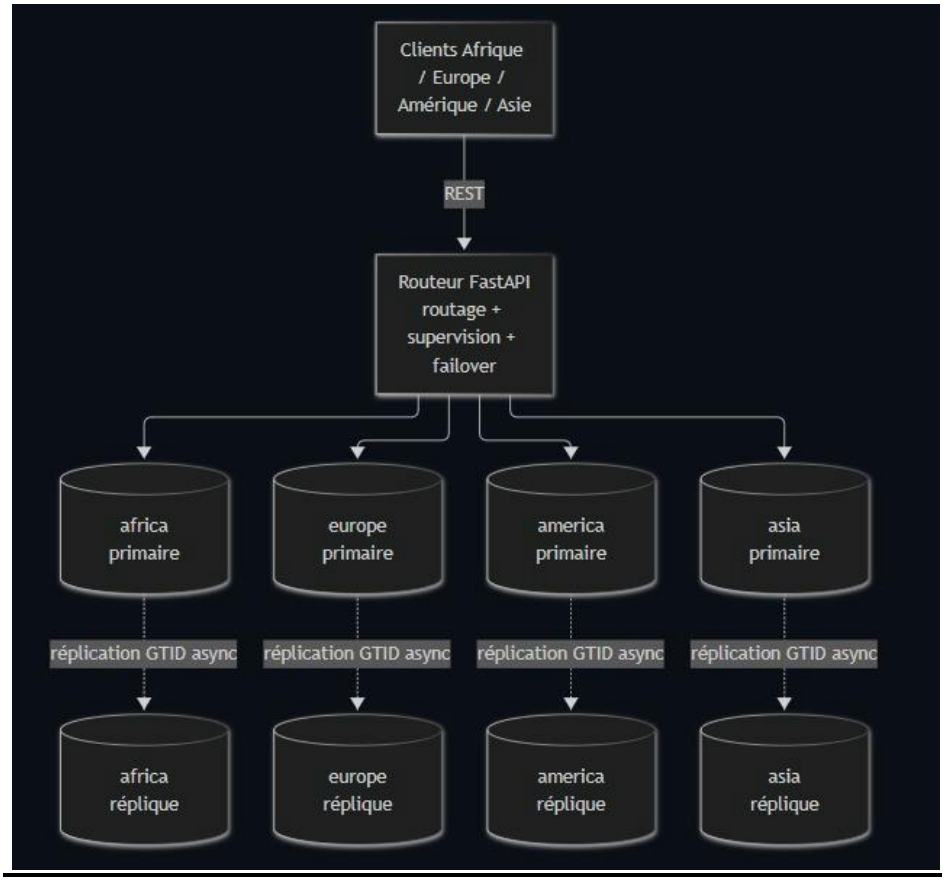


Figure 1 - Architecture réseau :



Flux de données

- **Requête client** : le client appelle l'API REST du routeur ; il ne connaît jamais les nœuds MySQL.
- **Routage** : le shard est déduit de l'ID ((id - 1) % 4), de la région (clients) ou du hash du nom (création de produit). Aucune table de correspondance n'est nécessaire.
- **Écritures** : toujours vers le **primaire** du shard propriétaire de la donnée.
- **Lectures** : primaire d'abord ; repli sur la réplique si le primaire est injoignable (réponse alors marquée served_by_replica). Le catalogue est lu en parallèle sur les 4 shards (scatter-gather).
- **Réplication** : asynchrone, par GTID, du primaire vers sa réplique.
- **Supervision** : une sonde par seconde sur chaque nœud ; état exposé par /cluster et /events.

Composant	Technologie	Rôle
Routeur	Python 3.12, FastAPI, mysql-connector	routage, transactions, lectures distribuées, supervision, failover
Nœuds de données	MySQL 8.4 (8 conteneurs)	stockage d'un shard ; primaire (écriture) ou réplique (copie)
Réseau	réseau Docker globalshop	interconnexion ; résolution de noms par service
Déploiement	Docker Compose	cluster complet en une commande

3. Stratégie de partitionnement

Chaque entité est fragmentée horizontalement. La clé de partition est choisie selon l'usage dominant de l'entité.

Entité	Clé de partition	Justification
Clients	Région géographique	Localité : les données d'un client restent sur son continent ; l'inscription est routée directement.
Produits	Hash : crc32(nom normalisé) mod 4	Répartition uniforme du catalogue et de la charge, sans point chaud.
Commandes	Shard du **produit** commandé	Colocalisation avec le stock : décrément + commande + facture dans une seule transaction locale.
Factures	Shard de leur commande	Intégrité référentielle locale (clé étrangère) et jointure sans réseau.

Identifiants globalement uniques

Chaque shard utilise auto_increment_increment = 4 et auto_increment_offset = rang du shard (1 à 4). Les IDs ne se chevauchent jamais (Afrique : 1, 5, 9 ... ; Europe : 2, 6, 10 ...) et le shard propriétaire se **déduit de l'ID** : toute lecture par ID va directement au bon nœud, sans balayage. Cette règle s'applique aussi après un failover, car primaire et réplique d'un même shard partagent le même offset.

Alternatives écartées

- **Partition par intervalles d'ID** : simple, mais crée des points chauds (les nouveaux IDs frappent toujours le dernier shard).
- **Annuaire central (table de correspondance)** : souple, mais ajoute un point de défaillance unique et une lecture par requête.
- **Commandes sur le shard du client** : favorise la latence client, mais impose une transaction distribuée (2PC) pour chaque achat. Nous avons préféré la cohérence forte et la simplicité du stock.

Limite assumée : le hash modulo 4 impose de redistribuer les données si l'on ajoute un shard. Un hachage cohérent (anneau de hash) serait la piste pour une élasticité réelle.

4. Modèle de données

Le même schéma est créé sur chaque nœud (init/01-schema.sql). Les clés étrangères existent à l'intérieur d'un shard (commande vers produit, facture vers commande) ; la référence client_id d'une commande pointe vers un client d'un autre shard et ne peut donc pas être une clé étrangère : elle est vérifiée par le routeur à la création de la commande.

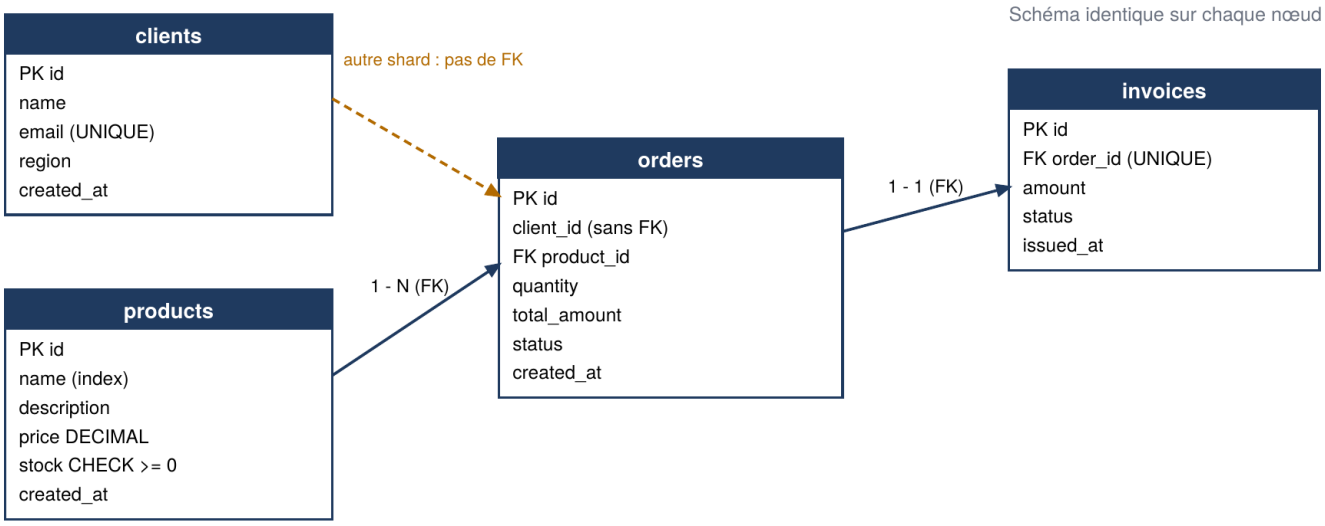


Figure 2 - Schéma relationnel d'un shard (diagramme entité-association)

La contrainte CHECK (stock >= 0) constitue une dernière barrière au niveau du SGBD. L'unicité de l'email est garantie par nœud (UNIQUE) et par le routeur à l'échelle du cluster (vérification sur les 4 shards, au mieux).

5. Réplication et tolérance aux pannes

5.1 Modèle de réplication

Chaque shard fonctionne en **primaire-réplique** avec une réplication MySQL **asynchrone par GTID** (SOURCE_AUTO_POSITION = 1). Les GTID identifient chaque transaction de manière unique, ce qui permet de rattacher un nœud à un autre sans calculer de position de binlog. Il n'y a **qu'un seul écrivain par shard** à tout instant : pas de multi-maître, donc aucun conflit d'écriture à résoudre. Le routeur configure lui-même la réplication au démarrage, puis la vérifie à chaque cycle de supervision.

5.2 Failover automatique

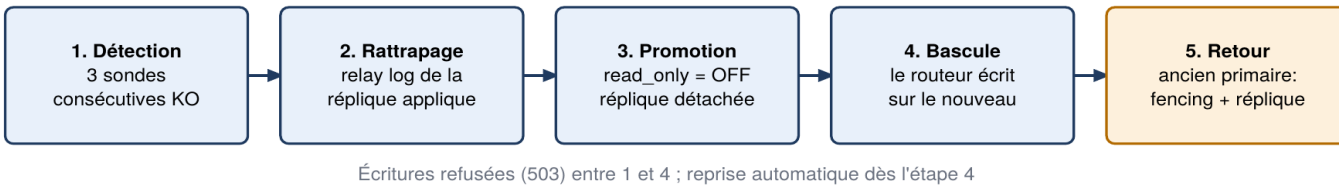


Figure 3 - Séquence de failover d'un shard

- **Détection** : après FAILOVER_THRESHOLD = 3 sondes consécutives en échec (1 sonde/s, délai maximal 2 s), le primaire est déclaré perdu.
- **Rattrapage** : la réplique cesse de recevoir, applique tout son relay log (limite le nombre de transactions perdues), puis est détachée de l'ancien primaire.
- **Promotion** : read_only passe à OFF ; le routeur bascule son routage. Les écritures reprennent sans intervention humaine.
- **Fencing** : lorsque l'ancien primaire revient, il est immédiatement passé en lecture seule puis rattaché comme réplique du nouveau primaire. Un seul écrivain existe à tout moment : pas de split-brain.
- **Cas NEEDS_RESYNC** : si l'ancien primaire détient des transactions validées mais jamais répliquées, MySQL refuse son rattachement. Le shard reste exploitable (nouveau primaire) mais sans redondance, jusqu'à la resynchronisation du nœud (procédure documentée dans le README).

5.3 Objectifs de reprise

Indicateur	Valeur	Commentaire
RTO (temps de reprise)	quelques secondes	détection (3 sondes) + rattrapage + promotion ; mesure dans la section 9
RPO (perte de données)	> 0 possible	réplication asynchrone : une transaction validée juste avant le crash peut ne pas être sur la réplique
Redondance après failover	perdue temporairement	le shard n'a plus de réplique tant que l'ancien primaire n'a pas été rattaché ou resynchronisé

6. Analyse CAP / PACELC

Positionnement : CP. Pour un stock, l'incohérence est plus coûteuse que l'indisponibilité : vendre deux fois le dernier article crée une commande impossible à honorer. Quand un primaire est injoignable, son shard **refuse les écritures** (HTTP 503) tant que le failover n'est pas terminé. Les trois autres shards continuent de servir normalement : l'indisponibilité est **isolée par shard**.

Situation	Écritures	Lectures	Cause / garantie
Nominal	primaire	primaire (cohérentes)	cohérence forte (EC)
Primaire HS (avant failover)	refusées (503)	réplique, marquées possiblement en retard	choix CP
Réplique HS	normales	primaire	shard sans redondance (statut DEGRADED)
Shard entier HS	refusées pour ce shard	résultat partiel signalé (partial)	isolation des pannes
Routeur HS	indisponibles	indisponibles	point de défaillance unique (limite, section 10)

PACELC : PC/EC. En cas de partition (P), nous choisissons la cohérence (C) plutôt que la disponibilité (A). En fonctionnement normal (E), nous choisissons encore la cohérence plutôt que la faible latence (L) : le stock est toujours lu et écrit sur le primaire avec un verrou de ligne, et la réplication asynchrone évite en revanche d'attendre un acquittement distant à chaque écriture (compromis latence/durabilité discuté en 10). Un choix AP (acceptation d'écritures divergentes puis réconciliation) a été écarté : il rendrait la survente possible.

7. Cohérence et anti-survente

Question du sujet : si deux clients achètent le dernier article en même temps sur deux nœuds différents, comment éviter la survente ? **Réponse** : un produit n'existe en écriture que sur **un seul primaire** (celui de son shard). Les deux achats, quel que soit le point d'entrée, passent par le routeur et arrivent sur ce même nœud, où le verrou de ligne les sérialise.

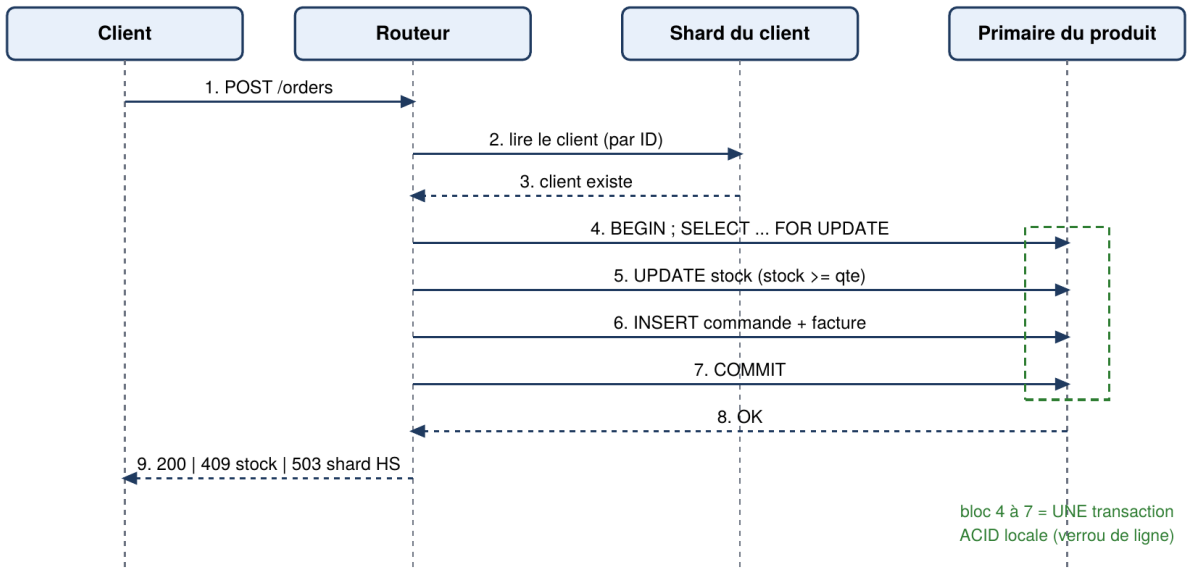


Figure 4 - Diagramme de séquence d'une commande (POST /orders)

Trois barrières successives

- **Verrou de ligne** : SELECT ... FOR UPDATE bloque les autres acheteurs du même produit jusqu'au COMMIT ; chacun voit le stock réel.
- **Mise à jour conditionnelle** : UPDATE ... SET stock = stock - q WHERE stock >= q ; le nombre de lignes modifiées est contrôlé.
- **Contrainte SQL** : CHECK (stock >= 0) interdit tout stock négatif, même en cas de bug applicatif.

Le décrément de stock, la commande et la facture sont dans **une transaction ACID locale** : tout est validé ou tout est annulé. Comme le stock et la commande sont colocalisés, **aucune transaction distribuée (2PC/XA) n'est nécessaire**, ce qui évite ses coûts (verrous longs, blocage en cas de panne du coordinateur). Le seul accès inter-shards d'une commande est une **lecture** (vérification du client). Les verrous sont toujours pris sur une seule ligne d'un seul nœud : pas d'interblocage possible entre nœuds.

Sémantique des réponses : 200 = commande validée ; 409 = stock insuffisant ; 503 = shard indisponible. Si la connexion est perdue pendant une écriture, la réponse 503 porte outcome: unknown : le client sait que le résultat est incertain et doit vérifier avant de réessayer.

8. Implémentation

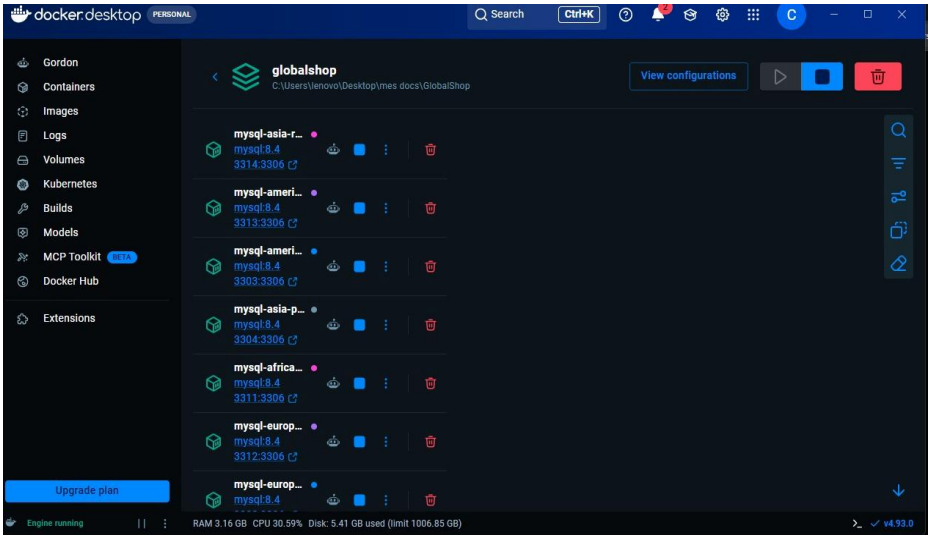
Élément	Détail
Dépôt	docker-compose.yml (cluster), init/01-schema.sql (schéma), router/main.py (routeur), router/static/index.html (tableau de bord), scripts/demo.py (démo et tests)
Déploiement	docker compose up -d --build ; attendre l'état OK des 4 shards sur /cluster

Élément	Détail
Fonctionnalités	inscription client ; création de produit ; catalogue distribué ; commande transactionnelle avec facture ; audit d'intégrité par produit
API principale	POST /clients, POST /products, GET /products, POST /orders, GET /orders/{id}, GET /health, GET /cluster, GET /events
Initialisation	le schéma est créé sans journalisation binaire (sql_log_bin = 0) : tous les nœuds partent d'un état identique, sans transaction locale parasite

9. Tests et résultats

9.1 Méthodologie

Les tests sont entièrement automatisés par scripts/demo.py et leurs mesures enregistrées dans *results/demo_results.json*. Environnement : Docker Desktop, cluster complet de 8 nœuds MySQL sur une seule machine



Les temps incluent donc la contention locale et ne représentent pas une latence réseau intercontinentale.

9.2 Test fonctionnel

Création d'un client par région, de 12 produits, lecture du catalogue distribué et passage d'une commande. Répartition des 12 produits par hash sur les shards et Produits lus dans le catalogue distribué :

Apres executions du script :

```
======================================================================
PARTIE 2 - Fonctionnalités : inscription, produits, catalogue distribué, commande
======================================================================
  [OK]   inscription client africa -> nœud africa (id 1)
  [OK]   inscription client europe -> nœud europe (id 2)
  [OK]   inscription client america -> nœud america (id 3)
  [OK]   inscription client asia -> nœud asia (id 4)
  [OK]   email unique à l'échelle du cluster (doublon sur un autre shard refusé)
  répartition des 12 produits par hash : {'asia': 3, 'europe': 3, 'america': 3, 'africa': 3}
  [OK]   IDs produits globalement uniques (offset/increment par shard)
  [OK]   shard déductible de l'ID : (id-1) % 4
  [OK]   catalogue distribué : 12 produits lus sur les 4 shards
  [OK]   lecture complète (aucun shard manquant)
  [OK]   commande : {'order_id': 4, 'invoice_id': 4, 'client_id': 1, 'product_id': 4, 'quantity': 2, 'total': 20.0,
'remaining_stock': 48, 'client_node': 'africa', 'product_node': 'asia'}
  [OK]   commande #4 + facture #4 sur le nœud asia
  [OK]   invariant stock : 48 restant + 2 vendu = 50
```

9.3 Test de charge : 100 achats concurrents

100 clients lancent un achat au même instant (barrière de synchronisation) sur un même produit dont le stock est de 20.

```
======================================================================
PARTIE 3.1 - Test de charge : 100 achats simultanés du même produit (stock = 20)
======================================================================
  produit #13 sur le nœud africa, client #6
  durée 2.32s -> 43 req/s | latence p50 1854 ms, p95 2178 ms, max 2195 ms
  commandes acceptées : 20 | refusées (stock épuisé) : 80 | autres : 0
  [OK]   exactement 20 achats acceptés (pas de survente)
  [OK]   80 achats refusés proprement (HTTP 409)
  [OK]   aucune erreur technique
  [OK]   stock final = 0, commandes en base = 20
```

9.4 Simulation de panne (chaos engineering)

20 écrivains envoient des commandes en continu sur un produit du shard cible. Après 5 s de régime nominal, le primaire du shard est tué brutalement (docker kill, SIGKILL) en pleine écriture. Le script observe la réponse du système, attend la reprise, relance l'ancien primaire et vérifie l'intégrité des données.

10. Limites et perspectives

- **Réplication asynchrone** : RPO > 0 possible. Piste : réplication **semi-synchrone** (une commande n'est acquittée qu'après réception par la réplique), au prix d'une latence d'écriture plus élevée, ou réplication à quorum (Group Replication / Raft, CockroachDB).
- **Une seule réplique par shard** : après un failover le shard n'a plus de redondance. Trois copies par shard permettraient de tolérer la perte de deux nœuds.
- **Routeur unique** : point de défaillance. Piste : plusieurs instances derrière un répartiteur de charge (le routeur est sans état hors supervision).

- **Commande mono-produit** : un panier multi-produits sur plusieurs shards demanderait une saga ou une transaction distribuée (XA/2PC).
- **Hash modulo 4** : l'ajout d'un shard impose une redistribution ; un hachage cohérent la limiterait.
- **Latence géographique** : le prototype simule les régions sur une seule machine ; en production, répliques de lecture dans chaque région.
- **Sécurité** : identifiants en clair dans le dépôt, acceptable pour un prototype local uniquement.

11. Conclusion

Le prototype répond aux trois objectifs : la **mise à l'échelle** par partitionnement en 4 shards avec des identifiants globalement uniques, la **tolérance aux pannes** par réplication et failover automatique, et la **cohérence du stock** par un écrivain unique, un verrou de ligne et des contraintes SQL. Le choix CP (PC/EC) assume un refus temporaire d'écriture sur un shard en panne plutôt qu'une incohérence. Les tests de charge et de chaos montrent le comportement réel du système, y compris ses limites (réplication asynchrone), ce qui constitue la base des améliorations décrites en section 10.